Manual and Technical Documentation

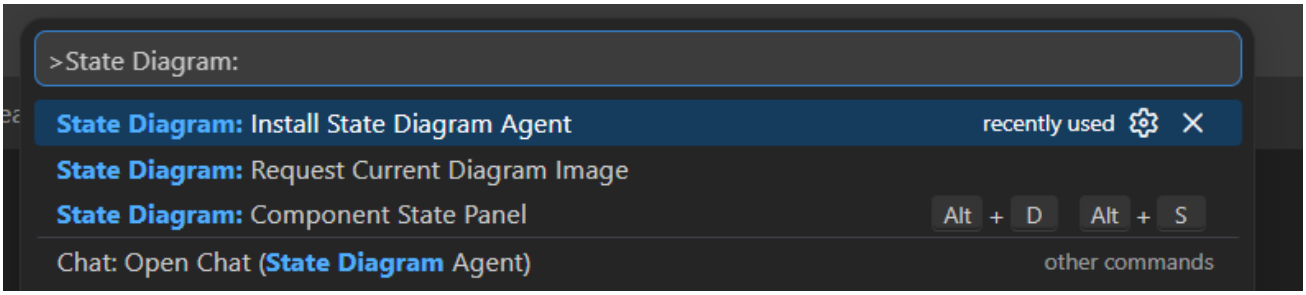
User Manual

React Components extension turn React component behavior into a clear, visual state flow diagram, for the VS Code editor, aiming for the best integration with existing ecosystem.

State Diagram feature highlights:

- Generated automatically from static code analysis (parser-based), no runtime required.
- Extracts states, transitions, guards, and mutation flow from React TypeScript/TSX components.
- Tracks common React state patterns, including useState and useRef, and supports configurable custom hook names.
- Understands control flow such as conditionals, loops, switches and optionally even early exits (return/throw).
- Integrates directly into VS Code, so you can inspect diagrams with Copilot without leaving the editor, allowing for easy documentation and refactoring

Commands



Accessible in a traditional VS Code way with **Ctrl+Shift+P** and prefixed with **State Diagram:**

> Component State Panel

Opens the **Component State panel** the place where the state diagram is displayed. For this to work the following has to be satisfied:

- Workspace has to be opened.
- There is a .tsx, .jsx, .ts, .js file currently focused in the editor and it contains *export default* with valid React component.

Can also be triggered with **Alt+D+S** shortcut by default.

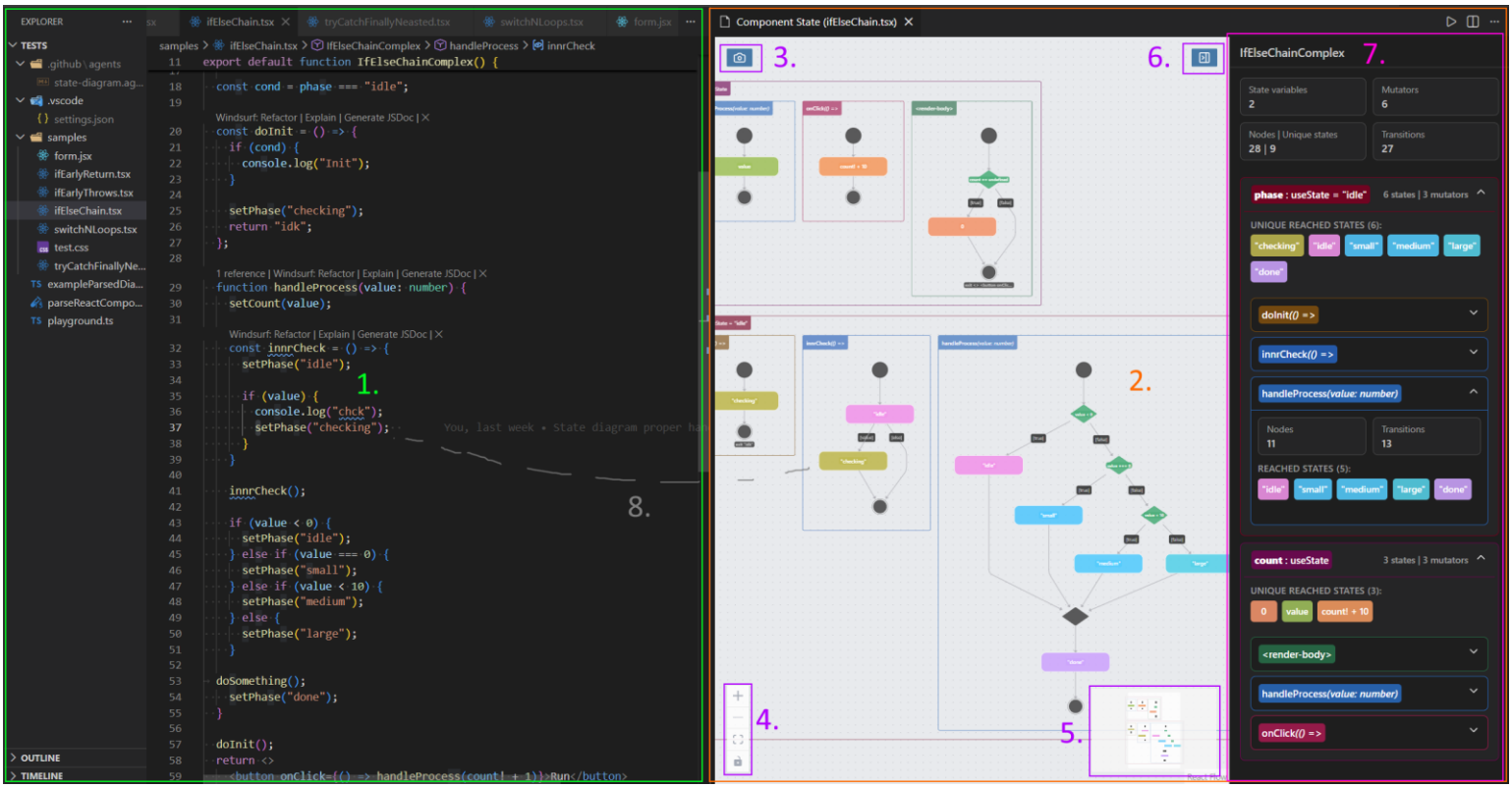
> Request Current Diagram Image

Providing that the **Component State panel** is opened, it will make a screenshot of current diagram and prompts you to select a directory where to place it.
Same thing can be achieved in the panel itself.

> Install State Diagram Agent

Will create **state-diagram.agent.md** in your workspace, ready to be used, more about that in AI Integration section.

User Interface

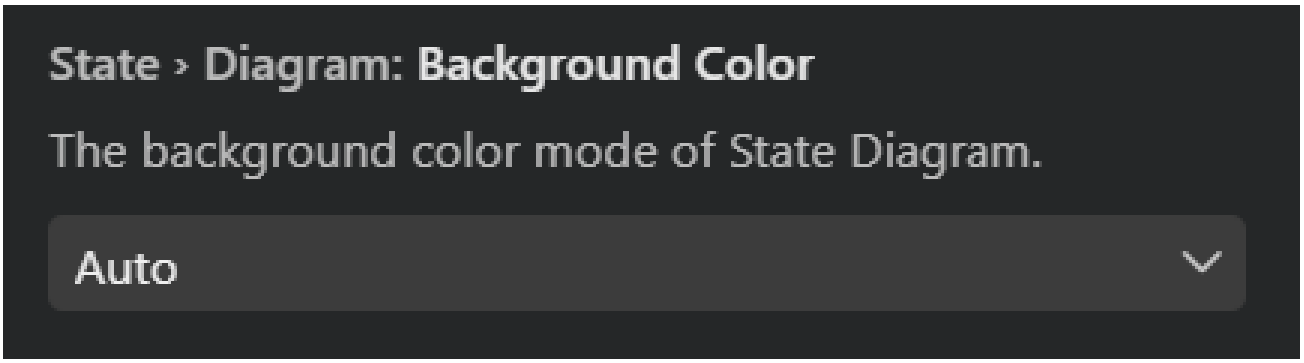


1. **The VS Code user interface –** Standard VS Code user interface with source code files.
2. **Component State panel** – The main UI where the state diagram is rendered.
3. **Screenshot** – Button that allows you to create image of the current diagram, equivalent **Install State Diagram Agent** command.
4. **Control Hud** – Basic buttons allowing you to zoom in/out, center the view, and allow/disallow moving it by dragging the elements.
5. **Minimap** – Miniature of the diagram for the ease of navigation when zoomed in.
6. **Trigger Overview** – Button that opens/closes Overview & Details menu.
7. **Overview & Details** – Side menu that shows you the overview of the state diagram with metrics like total number of state variables, mutators, states etc. Including the overview of reached states for each variable as well as individual mutator.
   Allows you to hide individual state variables, resulting in smaller diagram which affects Screenshot image as well as what AI integration obtains.
8. **Traceability feature** – When double-clicking a node it will open the relevant source file and place the cursor at the corresponding line of code the node is representing. Same will happen after clicking on the states inside the Overview & Details.

Settings

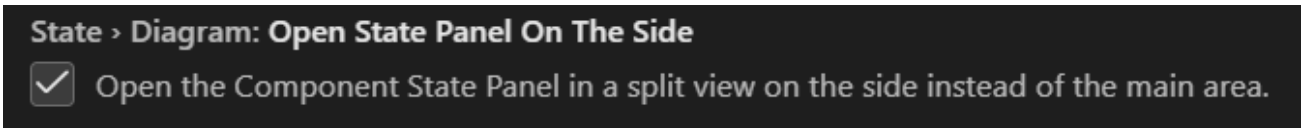
All the corresponding settings are located in VS Code settings under State > Diagram under Extensions > React diagrams.

Background Color Mode



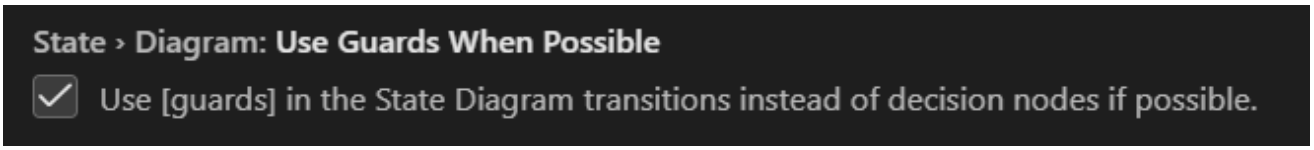
- **Auto (default)** - Uses the VS Code theme.
- **Force Light** - Use a white background for the state diagram.
- **Force Dark** - Use a dark background for the state diagram.

Open State Panel On The Side

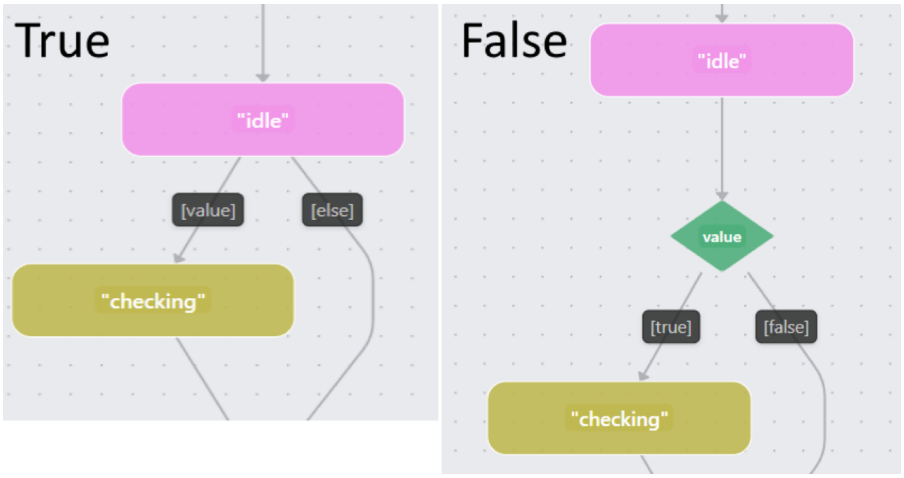


True/False – Default True

Prefer Guards When Possible

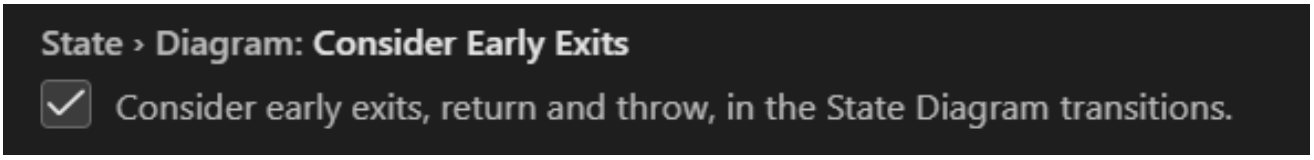


True/False – Default True



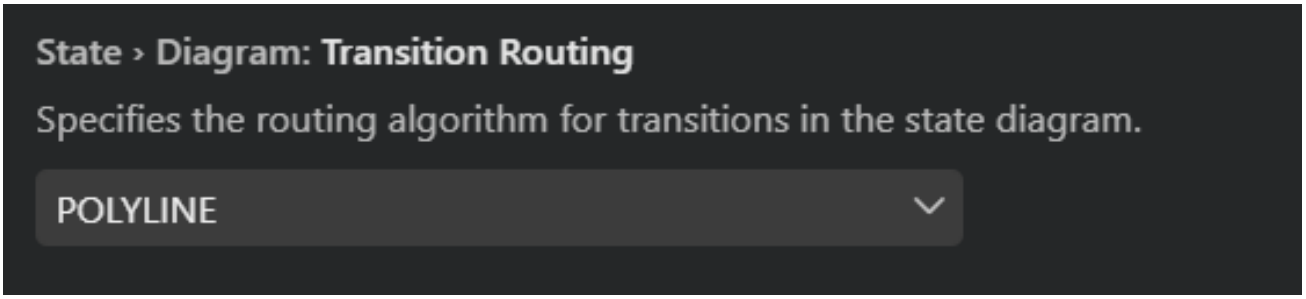
Can make diagram smaller in exchange of losing bit of explicitness.

Consider Early Exits



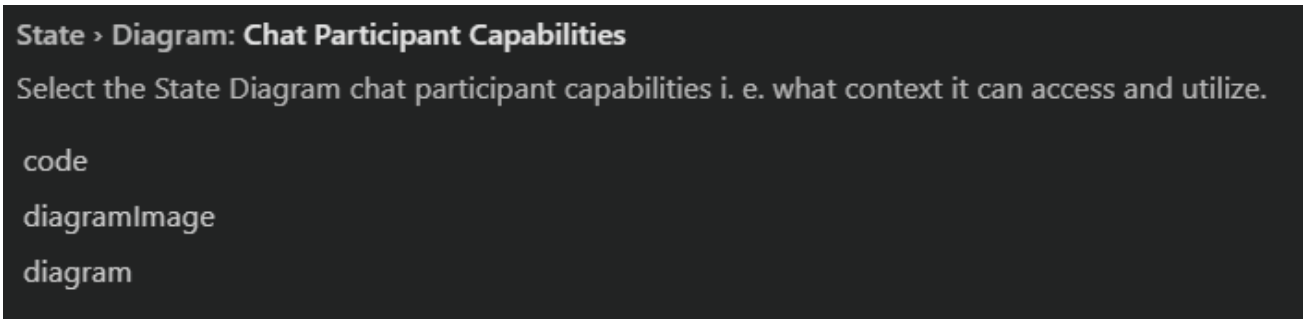
True/False – Default True

Transition Routing



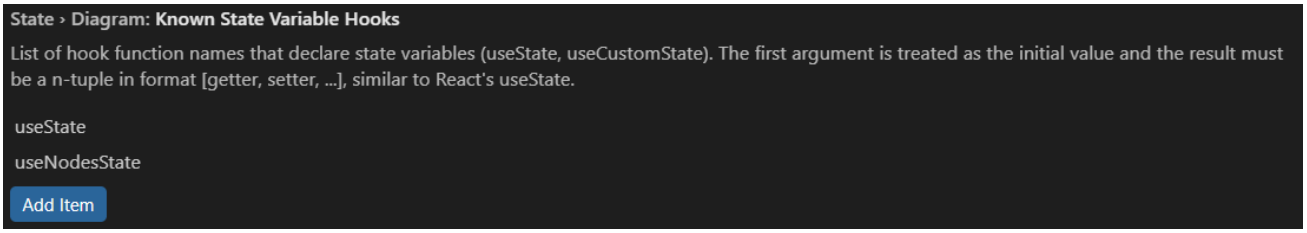
- **POLYLINE (default)** - Shorter transition with graceful turns.
- **ORTHOGONAL** - Longer transition with sharp turns.
- **SPLINES** - Combination of POLYLINE and ORTHOGONAL.

Chat Participant Capabilities



- **code (default)** - Enable participant to view current source file.
- **diagram (default)** - Enable participant to utilize serialized diagram representation.
- **diagramImage** - Enable participant to view diagram as image (Component State Panel has to be opened and visible for this to work).

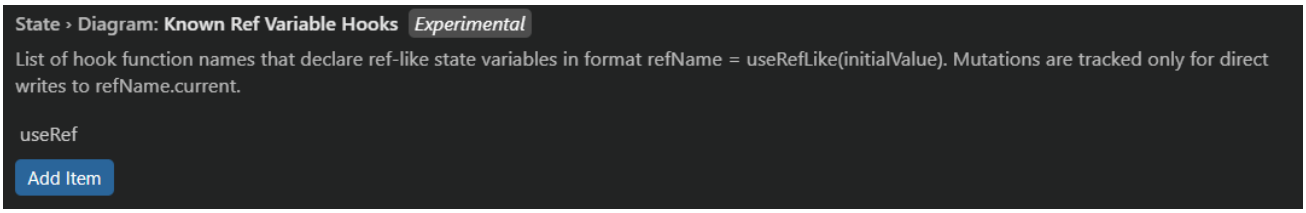
Known State Variable Hooks



Can take any value that satisfies the given criteria, default is **useState**.

Useful for hooks provided by third party libraries.

Known Ref Variable Hooks (experimental)

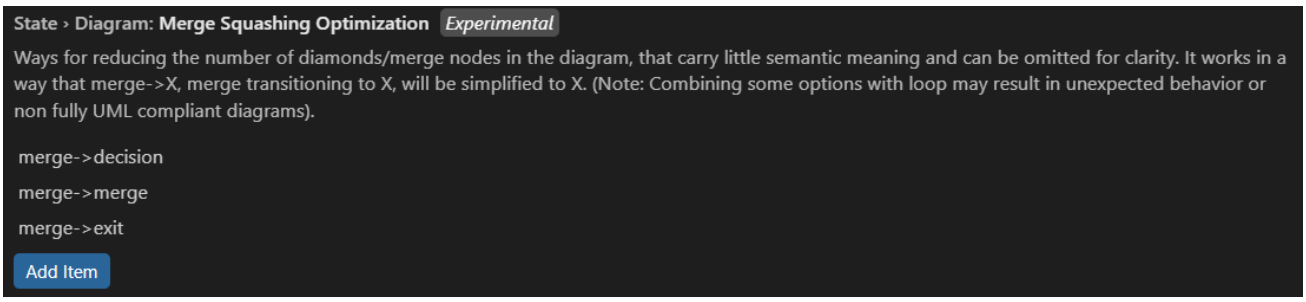


Can take any value that satisfies the given criteria, default is **useRef**.

Useful for hooks provided by third party libraries.

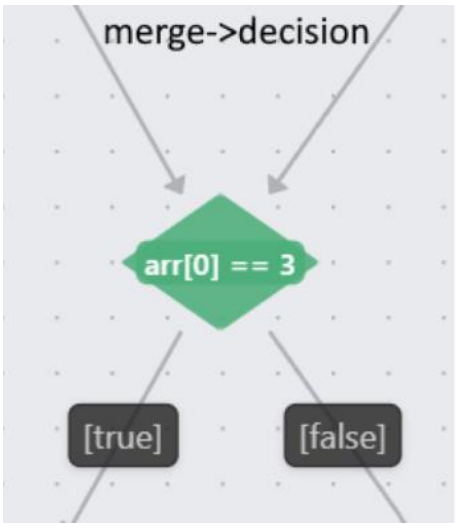
Experimental: This feature is experimental and will only work for direct mutations as described, it also may not work for ref binding to element.

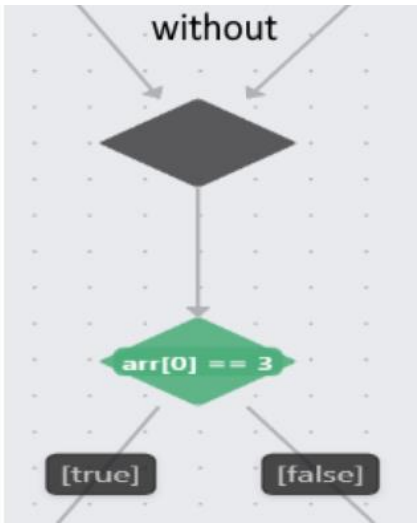
Merge Squashing Optimization (experimental)

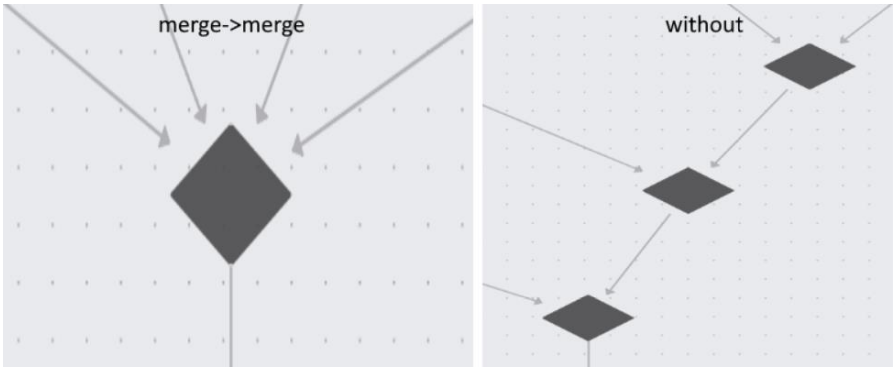


- **merge->merge (default)** – Squash merge nodes chains into a single merge node.
- **merge->decision (default)** – Emplace merge with the possible decision node.
- **merge->try (default)** – Emplace merge with the possible try node.
- **merge->switch** – Emplace merge with the possible switch node.
- **merge->loop** – Emplace merge with the possible loop node (do-while excluded).
- **merge->exit** – Emplace merge with the possible exit node.
- **do->x** – Emplace 'do' diamond with the following node.

Examples:





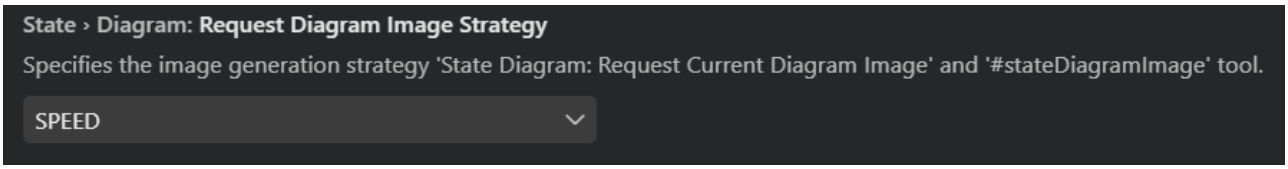


Can reduce the number of nodes and transition significantly for a certain type of diagrams.
Note that less nodes and transition also imply smaller context overhead for the AI model.

Experimental: As noted, some combinations of these can result in diagram potentially not being fully UML compliant as mentioned above, especially when combined with **Used Guards When Possible**.

Not that our aim is to provide diagrams that will be both reasonably understandable and reasonable in size by allowing these domain-specific rules, rather than strictly following to UML standards by all means necessary.

Request Diagram Image Strategy



- **ACCURACY (default) –** Prioritizes higher accuracy in diagram image generation, but a lot slower.
- **SPEED** – Prioritizes faster diagram image generation, with slight loss of accuracy.

Note that the this does not affect the Screenshot icon in the UI, but does affect the result when the result when **Request Current Diagram Image** is issued, as well as what the AI integration will receive.

Also note that **Known State Variable Hooks** and **Known Ref Variable Hooks** are specific to those kinds of hooks and will likely not automatically enable support for the global state store or external state management libraries like Zustand or Redux which are currently not directly supported.

Also note that for some visual based setting to take effect, **Component State Panel** will have to be restarted.

AI Integration

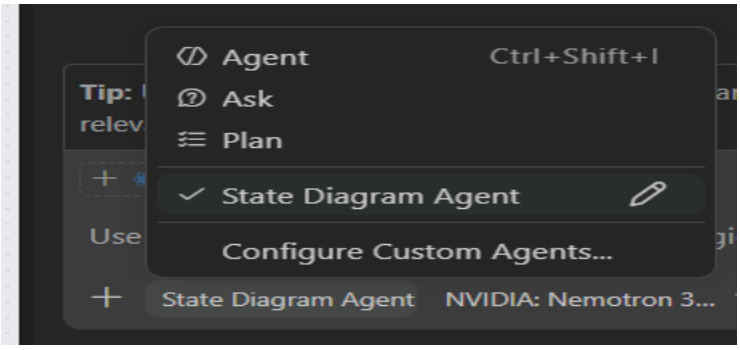
The extension integrates with VS Code Chat (i. e. “Copilot”) and Agents to provide diagram-aware AI assistance.

State Diagram Chat Participant

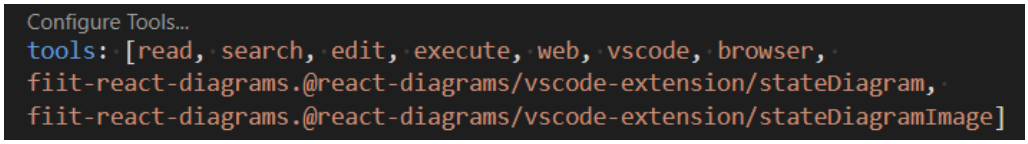
- How to use: Open chat and mention **@state-diagram** in the Chat window.
- Strengths: Build-in, present by default, fast to invoke, and more controllable through extension settings.
- Best for: One-shot/Fire-and-forget analysis, explanation, focused debugging, and single refactoring suggestions.
- Limitations: Cannot interact with the codebase directly, has little to no memory between isolated requests, and is not suited for long autonomous tasks.
- Context: Deterministic and configurable with **Chat Participant Capabilities** (code, diagram, diagramImage).

State Diagram Agent

- How to use:
    - Install: Run **Install State Diagram Agent** from the Command Palette. This will create the **state-diagram.agent.md** inside your *./.github/agents* folder and so it to you.
    - Use: Select the installed State Diagram Agent in Chat as the active agent.



- Strengths: True agentic workflow, it can decide when and what tools to call automatically, and can interact with the codebase as directly as needed.
- Best for: Larger, multi-step tasks, longer conversations and iterative workflows with memory of the running conversation.
- Trade-off: More automated but less predictable than one-shot participant since tool call strategy is agent-controlled.
- Context/tools: Can use code, diagram and diagramImage similarly to @state-diagram by calling tools (#stateDiagram, #stateDiagramImage) when available and when seen as fitting.
  Chat Participant Capabilities has no effect on the agent, it depends on his decision with respect to your prompt.
  Note that you can explicitly tell the agent to use the tools by tagging #stateDiagram or #stateDiagramImage. You can also explicitly disable the tools in question in the *.agent.md* file itself.



Participant vs Agent (quick comparison)

Aspect	State Diagram Chat Participant	State Diagram Agent
Availability	Built-in and always available	Must be installed first (**Install State Diagram Agent --> state-diagram.agent.md**)
How to invoke	Tag **@state-diagram** in Chat	Select agent in Chat
Typical flow	One-shot, fire-and-forget	Multi-step, iterative workflow
Codebase interaction	Limited/directly user scoped Can't edit files directly	Agent can operate across files as needed and call our respective tools.
Predictability	Usually more controlled	More autonomous, can be less predictable
Customizability	**Chat Participant Capabilities** setting	(Almost) Fully customizable by editing the *.agent.md.* (Slightly more prone to misconfiguration)
Memory style	Minimal to none	Uses ongoing chat/agent context
Best use case	Fast explanations and focused tasks, testing	Larger analysis/refactor workflows

Recommended usage pattern

1. Open the **Component State Panel** and verify the generated diagram.
2. Use **@state-diagram** in Chat for quick explanations, focused questions, or single refactoring ideas.
3. Switch to the State Diagram Agent when you want a multi-step, agentic workflow that can operate across the codebase.
4. When using agent, you do not have to attach the relevant files agent can get them automatically, but prefer to have them opened and focused, not changing between them. Both **@state-diagram** and the agent will prioritize what is currently shown in **Component State Panel.**

Notice

- Both Agent and Participant use Copilot and inherit most of its capabilities and possible flaws. Do not rely on them blindly, we provide absolutely NO warranty and carry absolutely zero liability.
- Keep capability settings aligned with your privacy and context needs.

Find the most up to date commands and settings in VS Code Command Palette and Settings (search for "State Diagram").